

School of Future Tech

ITM Skills University



Subject: Backend Development

on

Case Study 82: PageTurner Online Bookstore and Library Platform

Node.js, Express.js, MongoDB, JWT, Razorpay & Firebase

Full-Stack Web Application Development

by

Subrata Bhaktipad Panda — 150096725110

B.Tech Computer Science Engineering | Semester III

Cohort: Larry Page

Academic Year 2025–2029

Report

Node.js | Express.js | MongoDB & Mongoose | JWT & RBAC | Swagger | Postman | Render

Deployment Link : <https://pageturner-75gr.onrender.com>

Github Link : <https://github.com/subratapanda24/PageTurner.git>

Swagger Link : <https://pageturner-75gr.onrender.com/api-docs>

Index

No.	Section	Page
1	Abstract	3
2	Introduction	3
3	Objectives	3
4	Problem Statement	3
5	System Components and Requirements	4
6	Tools & Technologies	4
7	System Architecture and Folder Structure	5

No.	Section	Page
8	Requirement-to-Module Mapping	5
9	Database Architecture: MongoDB, Compass and Atlas	7
10	Authentication and Role-Based Access Control	8
11	Admin Book Upload and Media Pipeline	8
12	Payment Integration: Razorpay	10
13	Web User Interface	11
14	API Documentation using Swagger	12
15	API Testing using Postman	13
16	Deployment on Render	15
17	Testing and Verification Plan	15
18	Results and Conclusion	16

1. Abstract

PageTurner is a full-stack online bookstore and library platform built with Node.js, Express.js and MongoDB. It lets customers browse books by genre, search by keyword, read verified reviews, add titles to a shopping cart, save books to a wishlist, pay through a digital checkout and view their order history. Administrators get a separate management panel to create, update and delete books, upload cover images and control stock.

The project demonstrates core back-end and full-stack concepts: the Model-View-Controller (MVC) pattern, a REST API, Mongoose schemas and relationships, JWT-based authentication, bcrypt password hashing, role-based access control, multipart file upload with Multer, cloud storage with Firebase, payment verification with Razorpay HMAC-SHA256 signatures, interactive API documentation with Swagger (OpenAPI 3.0), API testing with Postman, and cloud deployment on Render with MongoDB Atlas. Progressive fallbacks (an in-memory database, local image storage and a payment sandbox) let the platform run in any evaluation environment.

2. Introduction

Online bookstores need more than a product list. They must identify users securely, keep a cart and wishlist per customer, take payments safely, protect administrator functions and stay available even when an external service is not configured. These are the same problems faced by most e-commerce systems, so a bookstore is a practical way to practise them together.

This report presents the architecture and implementation of PageTurner. Every feature is traced to a module and a test so that it shows what was built, why it was built that way and how it can be verified through three independent interfaces: the Web UI at /, the Swagger UI at /api-docs and the Postman collection at /api/postman-collection.

The design principles followed throughout are: keep routes, controllers, models and middleware in separate layers; hash and never store plain passwords; verify payments on the server and never trust the client; restrict privileged actions by role; and provide a fallback for every external dependency.

3. Objectives

- Catalogue and search: list books with pagination, filter by genre and search by title, author, genre or description.
- Authentication: register and log in users and protect routes with JWT bearer tokens.
- Role-based access: separate customer permissions from administrator permissions.
- Cart and wishlist: let each user manage a personal cart and wishlist.
- Checkout and payment: create orders, accept Razorpay payments and verify them on the server.
- Reviews: allow purchased-book reviews and show ratings on each book.
- Admin management and media: create, update and delete books and upload cover images with a cloud or local fallback.
- Documentation and testing: provide Swagger documentation and a Postman collection, and deploy the application on Render.

4. Problem Statement

An online book retailer needs a web platform where customers can discover and buy books and where staff can manage the catalogue and inventory. The platform must authenticate users, keep personal data such as carts and wishlists separate per user, process online payments safely, restrict catalogue changes to administrators and remain runnable even if the database, image storage or payment gateway is not configured.

Expected outcomes:

- A layered Express application following MVC, with Mongoose models for User, Book, Cart, Order and Review.
- A secure authentication and role-based authorisation system.
- An admin upload pipeline for book cover art and a two-step verified payment flow.
- Interactive API documentation, a Postman collection and a live deployment.

Scope. The project delivers a working web application and REST API. Payments run in Razorpay test mode or the built-in sandbox, and demo data (15 curated books and sample users) is created by a seed script.

5. System Components and Requirements

The table summarises the main components of the system.

Component	Technology	Detail
Web server	Node.js, Express.js	Routes requests, parses JSON, handles multipart uploads and mounts middleware.
Data models	MongoDB, Mongoose	Strict schemas for User, Book, Cart, Order and Review with validation and populate().
Authentication	JWT, bcrypt.js	Passwords hashed with 10 salt rounds; HMAC-SHA256 signed tokens with user ID and role.
Access control	Middleware (protect, adminOnly)	Separates standard user routes from admin-only routes; returns 403 for non-admins.

Component	Technology	Detail
Media upload	Multer, Firebase Admin SDK	5 MB image limit; stored in Firebase Storage or in public/uploads/ as a fallback.
Payments	Razorpay	Server-created orders and HMAC-SHA256 signature verification of each payment.
API documentation	Swagger (OpenAPI 3.0)	Interactive endpoint forms at /api-docs.
Fallback database	MongoMemoryServer	In-memory database started automatically when no external database is available.
Front end	HTML5, CSS, JavaScript	Dashboard in public/ with vanilla CSS design tokens and modular scripts.

6. Tools & Technologies

Category	Tool	Purpose
Runtime	Node.js	Asynchronous, event-driven server-side JavaScript.
Framework	Express.js	HTTP routing, middleware chain and static file serving.
Database	MongoDB / MongoDB Atlas	Document database; Atlas is the cloud-hosted option used on Render.
ODM	Mongoose	Schemas, validation, relationships and pre-save hooks.
Database viewer	MongoDB Compass	Desktop GUI to inspect and edit local or cloud data.
Security	jsonwebtoken, bcryptjs	Token signing and verification; password hashing.
File upload	Multer, Firebase Admin SDK	Multipart parsing and cloud image storage.
Payments	Razorpay	Order creation and payment signature verification.
API tools	Swagger UI, Postman	Documentation and testing of REST endpoints.
Hosting	GitHub, Render	Source control and cloud deployment of the web service.

7. System Architecture and Folder Structure

The project follows the Model-View-Controller pattern. A request enters the Express server, is matched by a route, passes through middleware for authentication, upload handling and validation, reaches a controller that contains the business logic, and uses a Mongoose model to read or write the database.

Layer / folder	Responsibility
src/server.js	Application entry point; mounts routes, static files and Swagger and starts the listener.
src/config/	Configuration for MongoDB (db.js), Firebase (firebase.js), Razorpay (razorpay.js) and Swagger (swagger.js).
src/models/	Data definitions for User, Book, Cart, Order and Review.
src/routes/	Express routers binding HTTP endpoints to controller methods.
src/middleware/	Authentication, validation and file upload handlers.
src/controllers/	Business logic: pricing, queries and order handling.

Layer / folder	Responsibility
src/utils/	Database seed script with curated books, covers and demo users.
public/	HTML5 dashboard, CSS design tokens and frontend JavaScript.

7.1 Request flow

```

Client (Browser / Postman)
  |
  v
Express Server (src/server.js) - CORS, JSON parser, static /uploads
  |
  v
Routes (src/routes/*.js)      - URL + HTTP method -> controller
  |
  v
Middleware (src/middleware/*.js) - auth, upload, validation
  |
  v
Controllers (src/controllers/*.js)
  |
  v
Models (src/models/*.js)      - Mongoose schemas
  |
  v
Database (MongoDB / MongoMemoryServer)

```

Figure 7.1 — MVC request flow of PageTurner

8. Requirement-to-Module Mapping

Each objective of the project is handled by a specific module. The table links requirement, module, component and concept.

#	Requirement	Module	Main component	Concept
1	Browse and search books	Catalogue	Book controller, Book model	Pagination, filters, keyword search
2	Register and log in	Authentication	Auth controller, User model	JWT, bcrypt
3	Manage cart and wishlist	Cart / Wishlist	Cart controller, Cart model	populate(), references
4	Place and pay for orders	Orders	Order controller, Razorpay config	HMAC-SHA256 verification
5	Review purchased books	Reviews	Review controller, Review model	Purchase check, ratings
6	Admin book management	Admin panel	protect, adminOnly	Role-based access control
7	Upload cover images	Media	uploadMiddleware, Firebase config	Multer, cloud storage
8	Document the API	Swagger	swagger.js	OpenAPI 3.0
9	Run without external services	Fallbacks	db.js, firebase.js, razorpay.js	MongoMemoryServer, local storage

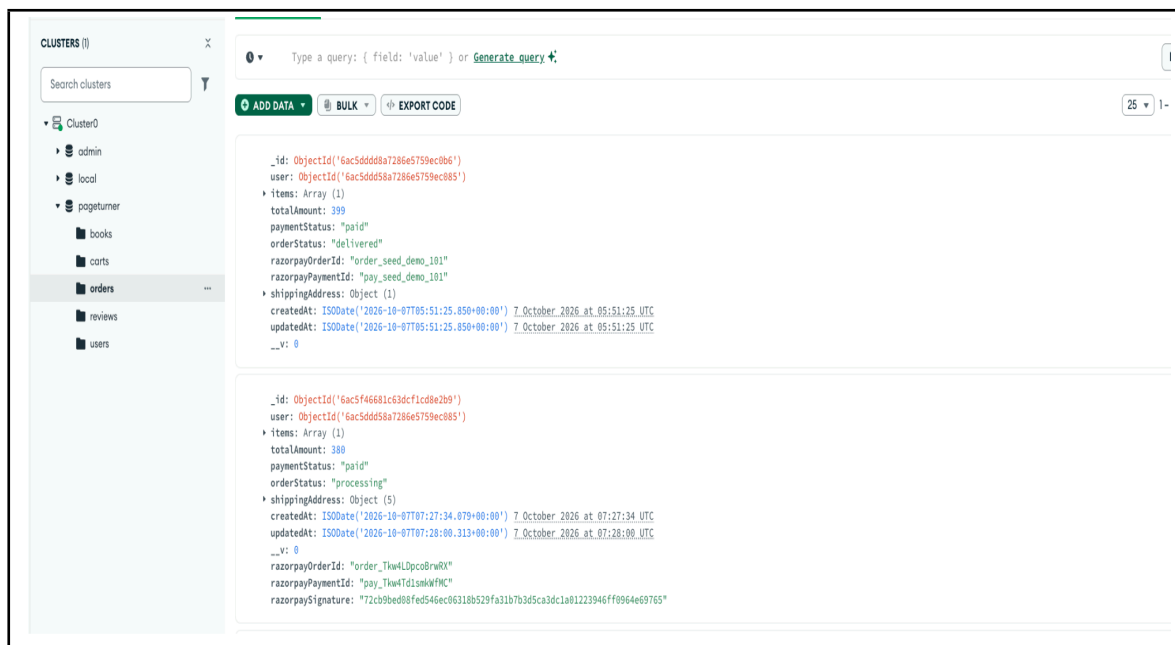
9. Database Architecture: MongoDB, Compass and Atlas

Understanding the difference between MongoDB, Compass and Atlas is important for evaluation and deployment.

Component	Description
MongoDB	The database engine that stores collections of documents (Users, Books, Orders, Reviews). Locally it listens on mongodb://127.0.0.1:27017. If neither a local nor a cloud database is available, the application starts mongodb-memory-server, an isolated in-memory database.
MongoDB Compass	A desktop GUI installed on the developer's computer. It does not store data; it connects to a local or cloud database so documents can be viewed, edited and queried visually.
MongoDB Atlas	MongoDB hosted on cloud servers. Render cannot reach the laptop's local 127.0.0.1 address, so a cloud database provides a mongodb+srv connection string reachable from both Render and the developer's machine.

Compass can be connected to the same Atlas connection string to view live cloud records from a laptop.

9.1 Screenshots



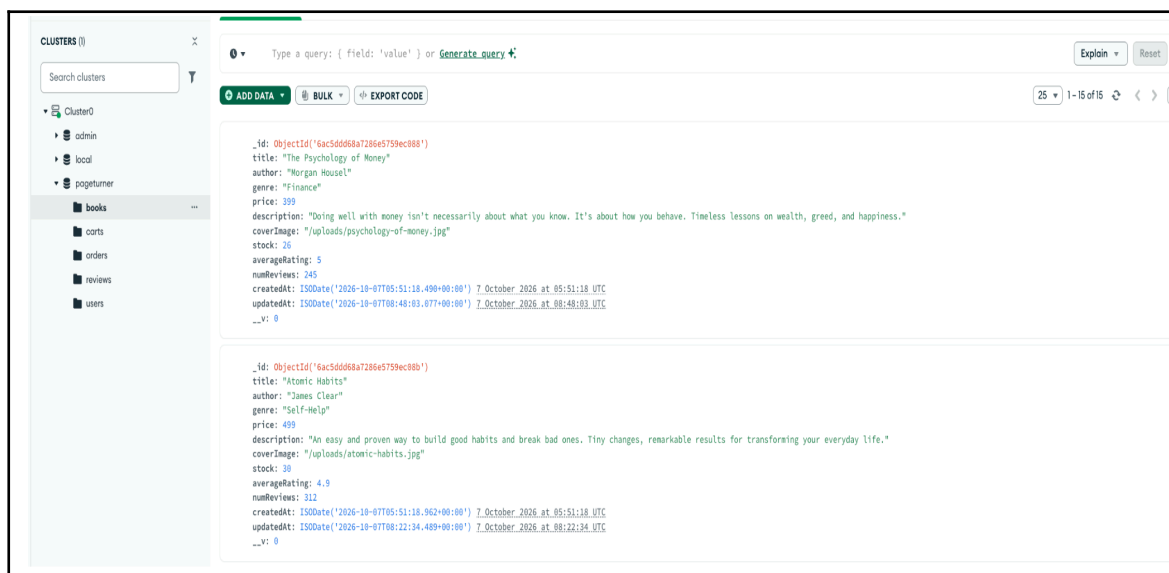
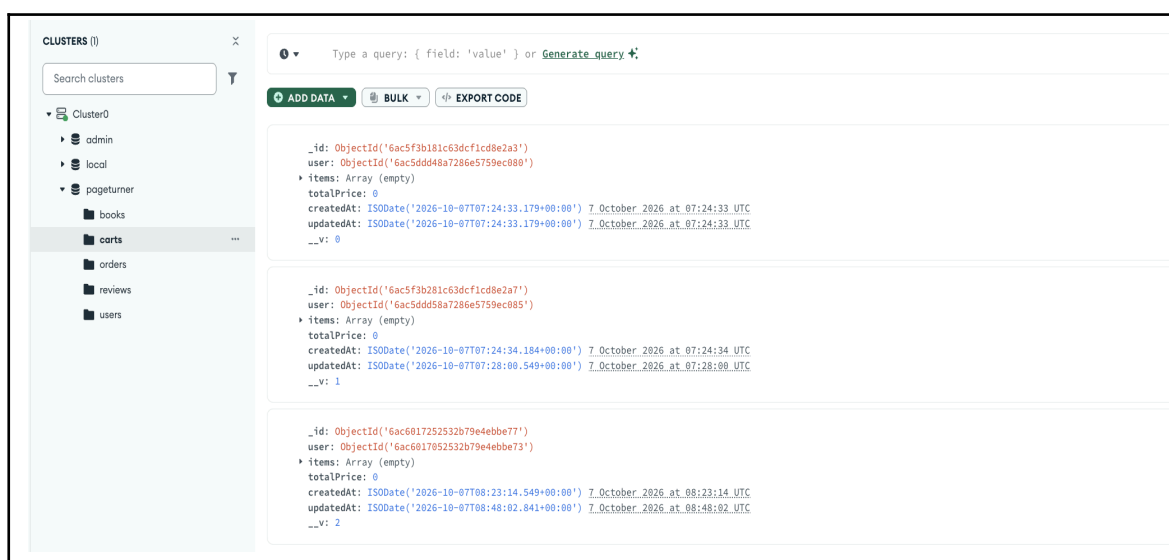


Figure 9.2 — Collections and documents (Books / Users / Orders)



10. Authentication and Role-Based Access Control

10.1 Registration and login

On registration the client sends name, email and password. The controller checks that the email is unique, a Mongoose pre-save hook hashes the password using bcrypt with 10 salt rounds, the user is stored, and the server signs a JWT containing the user ID and role with JWT_SECRET (30-day expiry). On login the controller fetches the user by email, runs bcrypt.compare on the entered password and returns a fresh token if it matches.

10.2 Protected and admin routes

Middleware	Behaviour
protect	Reads the Authorization header in the form Bearer <token>, verifies it with jwt.verify, finds the user and attaches it to req.user.
adminOnly	Checks req.user.role === "admin". A standard user receives HTTP 403 Forbidden (Not authorized as an admin); an admin continues to the controller.

Route protection is expressed as middleware chains: public routes use none, customer routes use protect, and admin routes use protect followed by adminOnly.

11. Admin Book Upload and Media Pipeline

Administrators can upload cover art together with book details. The request is sent as multipart/form-data to POST /api/books.

- Multer reads the file into memory (memoryStorage) with a 5 MB limit and accepts only image/jpeg, image/png and image/webp.
- If Firebase is configured, the buffer is written to a Firebase Storage bucket and a public download URL is stored.
- If Firebase credentials are missing, the file is saved to public/uploads/ and the relative path (for example /uploads/book-123.jpg) is stored.
- The book document is saved in MongoDB with title, author, genre, price, stock, description and cover image.

11.1 Screenshot

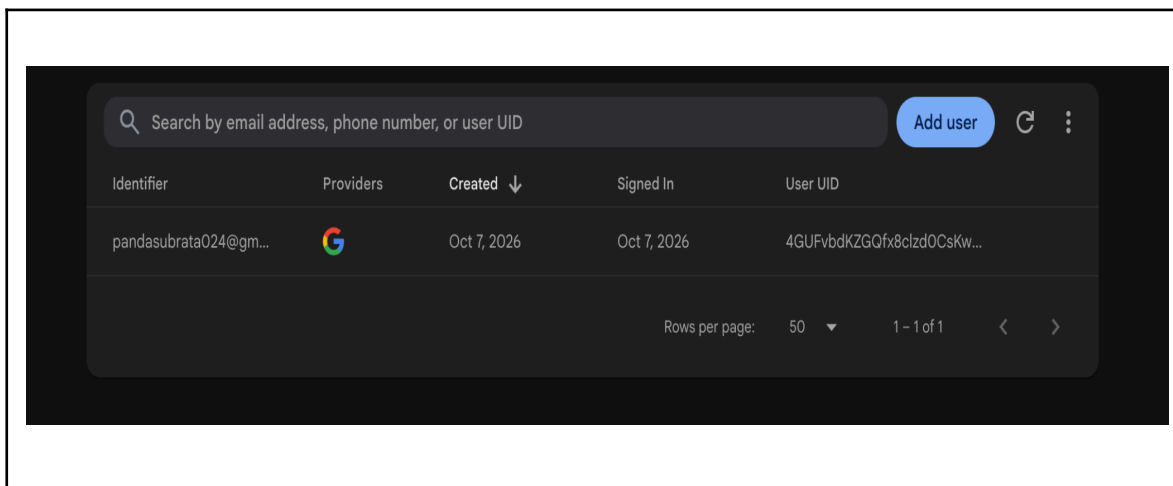


Figure 11.1 — Users data stored in firebase

12. Payment Integration: Razorpay

A two-step verification flow prevents fraudulent changes to order status.

Step	Action
1. Order creation	Client calls POST /api/orders with a shipping address. The server calculates the cart total, requests a Razorpay order (amount in paise, amount x 100), saves a local Order with paymentStatus = pending and returns the order ID and razorpay_order_id.
2. Client payment	The Razorpay Checkout modal collects payment and returns razorpay_order_id, razorpay_payment_id and razorpay_signature.
3. Server verification	Client calls POST /api/orders/verify. The server recomputes the signature and compares it with the received one. On a match the order becomes paid, the cart is cleared and book stock is reduced; otherwise HTTP 400 is returned.

12.1 Signature verification

```
const hmac = crypto.createHmac("sha256", process.env.RAZORPAY_KEY_SECRET);
hmac.update(razorpay_order_id + "|" + razorpay_payment_id);
const expectedSignature = hmac.digest("hex");
// order is marked paid only if expectedSignature === razorpay_signature
```

12.2 Screenshot

The screenshot displays the Razorpay Payments dashboard. At the top, there are tabs for 'Payments' and 'Orders'. Below the tabs, there are filters for 'All', 'Created', 'Captured', 'Refunded', and 'Failed'. A search bar and a dropdown for 'in Payment ID' are also present. Below the filters, there is a table showing a list of payments. The table has columns for Payment ID, Bank RRN, Customer detail, Created on, Amount, and Status. All payments shown are in 'Captured' status.

Payment ID	Bank RRN	Customer detail	Created on	Amount	Status
pay_TkxQtXDraqnZS8	--	+91 7977 175792 pandasubrata024@gmail.com	Wed Oct 7, 2:17pm	₹ 399.00	Captured
pay_Tkx2GMdXYLxbLH	--	+91 7977 175792 pandasubrata024@gmail.com	Wed Oct 7, 1:54pm	₹ 399.00	Captured
pay_Tkw4Td1smkWfMC	--	+91 7977 175792 user@pageturner.com	Wed Oct 7, 12:57pm	₹ 380.00	Captured
pay_Tku76gwhRVQTK	--	+91 7977 175792 user@pageturner.com	Wed Oct 7, 11:02am	₹ 598.00	Captured
pay_Tkttiga1ulV6nH	--	+91 7977 175792 user@pageturner.com	Wed Oct 7, 10:50am	₹ 499.00	Captured
pay_Tkl7hnnBoQL6PM	--	+91 7977 175792 user@pageturner.com	Wed Oct 7, 2:15am	₹ 480.00	Captured

Figure 12.1 — Razorpay checkout / payment flow

✓

₹ 399.00

Captured

Created on: Wed Oct 7, 02:17pm

Details

Payment ID

pay_TkxQtxDraqnZS8

Bank RRN

--

Order ID

order_TkxQIdOLFRL7uU

Invoice ID

--

Payment method

Net banking (CNRB bank)

Customer details

+91 7977 175792

pandasubrata024@gmail.com

Total Fee

₹ 9.42

Razorpay Fee - ₹ 7.98

GST - ₹ 1.44

This fee could've been reduced with Cashback on RazorpayX Corporate Card

Apply Now

Fee bearer

You pay the Razorpay platform fee

App Name

--

App ID

--

Description

Book Order Checkout

Notes

--

Timeline

✓ Payment created

Wed, Oct 7, 2026 2:17 PM

✓ Payment authorized

Wed, Oct 7, 2026 2:17 PM

✓ Payment captured

Wed, Oct 7, 2026 2:17 PM

✓ Settlement (Processed)

Fri, Oct 9, 2026 1:09 PM

ID: setLTjJLCwcJRIB1g

UTR number: db49lb0vjkc712em81g

Net amount: ₹ 2,689.98

View details

3 Initiate full, partial, or instant refunds to your customers

Issue refund

Help & Support

13. Web User Interface

The web dashboard is served from the public/ folder at the root URL. Customers browse and buy books; administrators manage the catalogue.

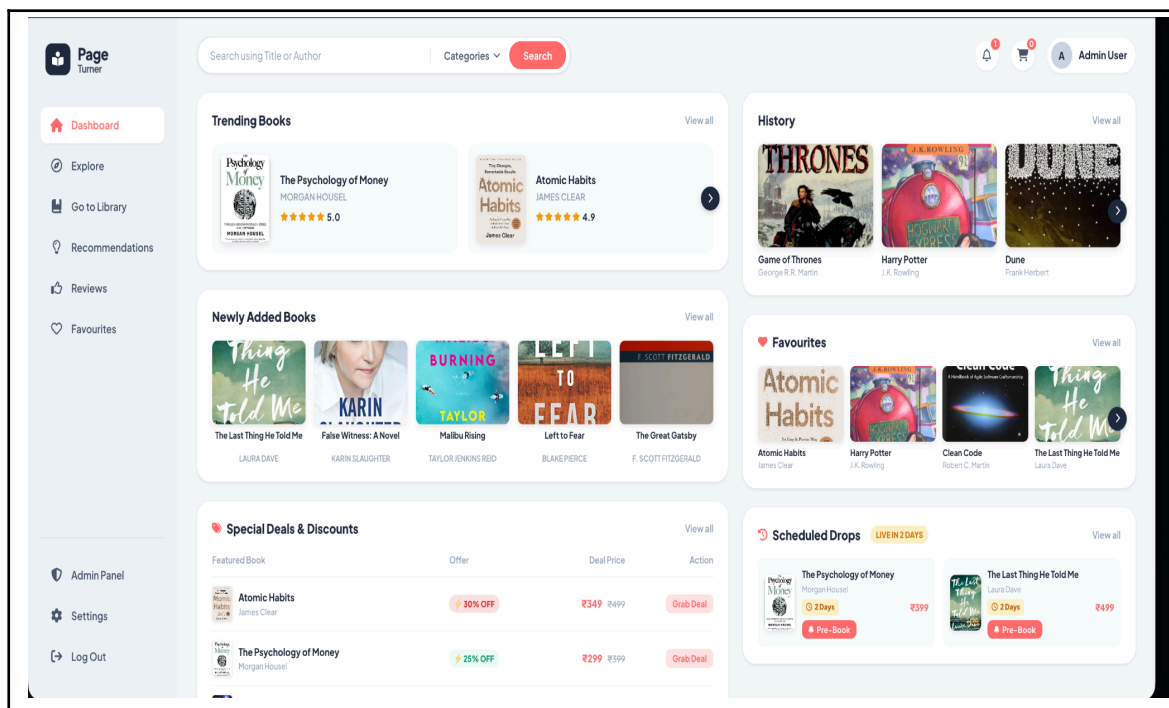


Figure 13.1 — Home page / dashboard of PageTurner

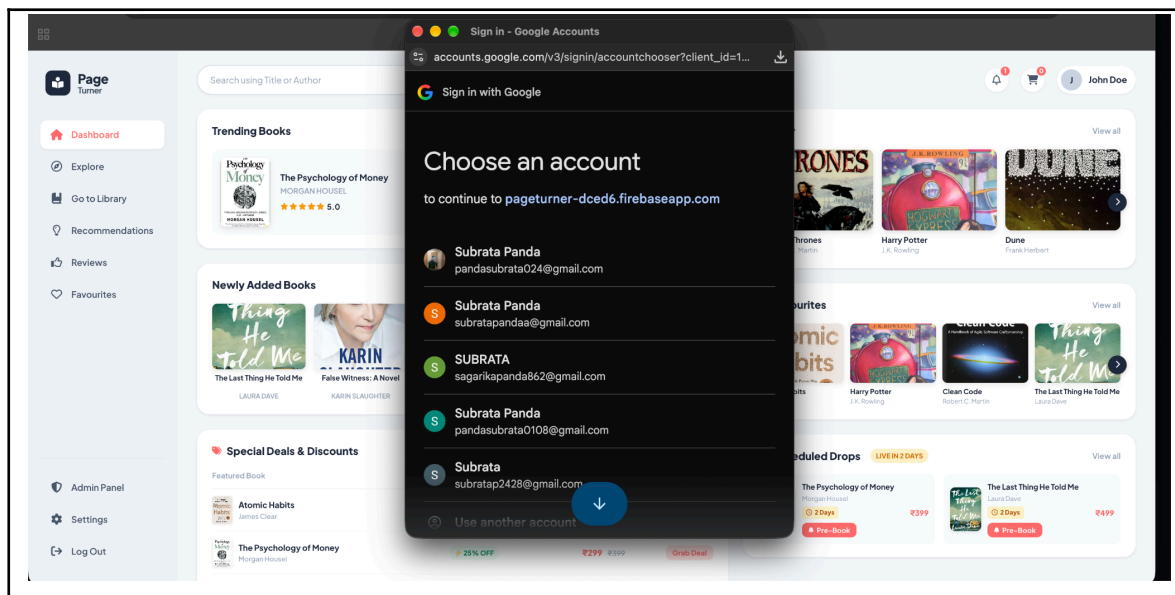


Figure 13.2 — Firebase authentication

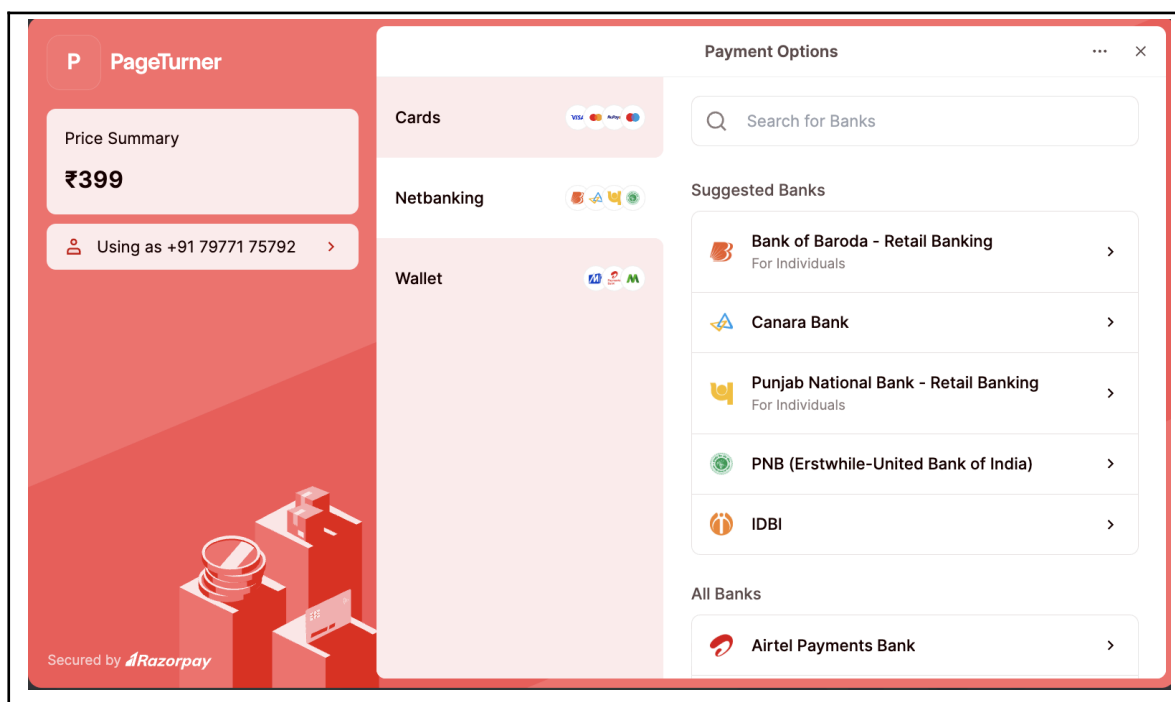


Figure 13.3 — Mode of payment options

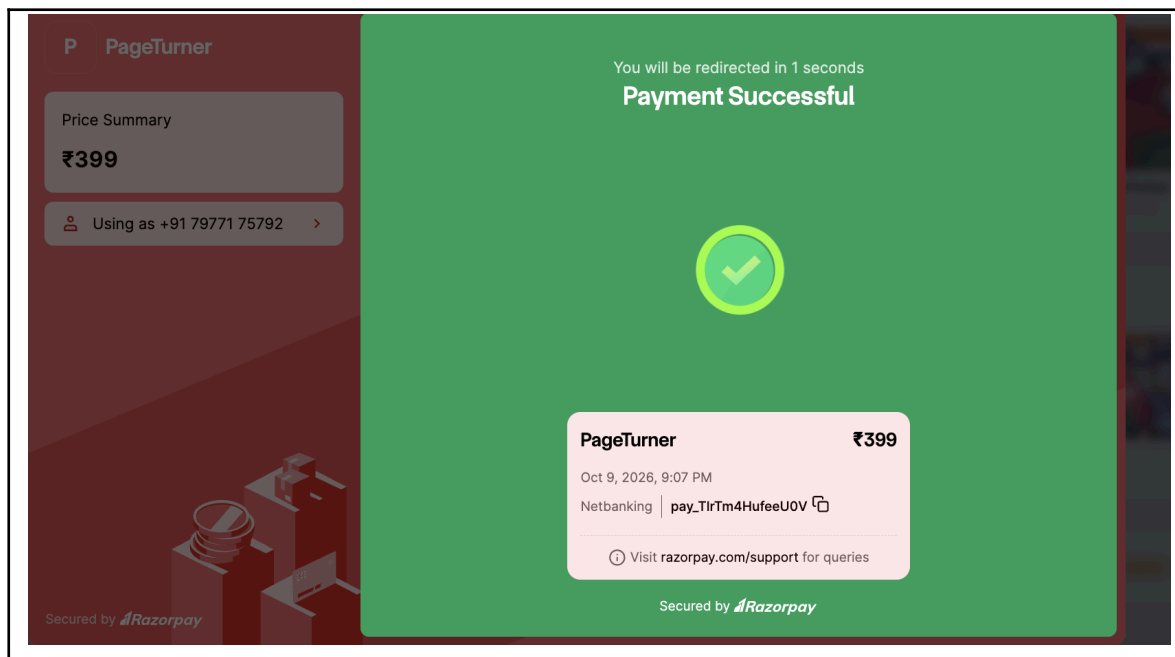


Figure 13.4 — Payment successfull

PageTurner Admin Manager

Upload book covers (Firebase Storage) and maintain books catalog.

TITLE

AUTHOR

GENRE / CATEGORY

PRICE (₹)

STOCK

Fiction

10

Please fill in this field.

DESCRIPTION

COVER FILE (FIREBASE UPLOAD)

OR IMAGE URL

Choose file No file chosen

https://...

Save Book

Clear

Current Catalog Items

	How to Stop Worrying and Start Living Dale Carnegie • ₹399	Delete
	The Notebook Nicholas Sparks • ₹350	Delete

Figure 13.5 –.Admin Pannel

14. API Documentation using Swagger

Swagger UI lists every backend route with its HTTP method, parameters, request body, authentication requirement and response, and lets the examiner test the API directly from the browser at /api-docs. Endpoints are grouped as Health, Books, Auth, Cart, Wishlist, Orders and Reviews.

14.1 Demo order

#	Step
1	Open /api-docs and run GET /api/health.
2	Run GET /api/books and copy a book _id.
3	Run POST /api/auth/login with the demo user and copy the returned token.
4	Click the green Authorize button and paste the token (with or without "Bearer", depending on the prompt).
5	Test protected routes: cart, wishlist, orders and reviews.
6	Log in as admin, authorize with the admin token and show the book create, update and delete routes.

14.2 Screenshots

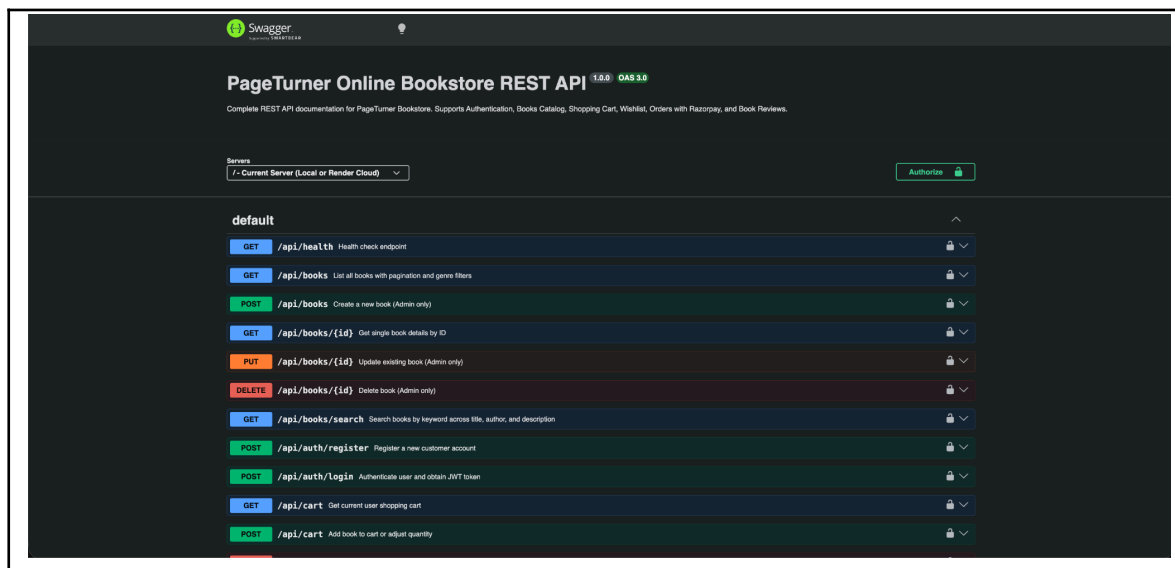
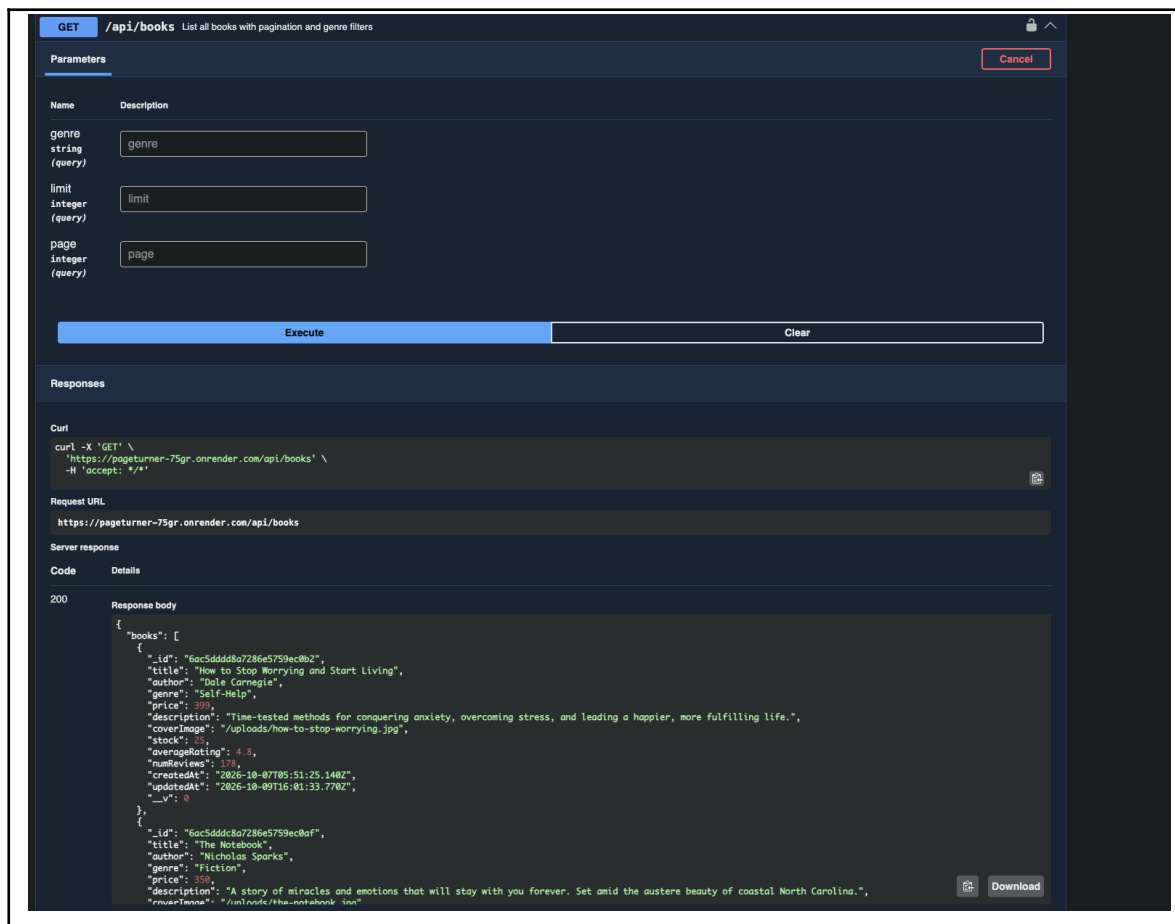


Figure 14.1 — Swagger UI endpoint list at /api-docs



15. API Testing using Postman

The collection `postman_collection.json` (also available at `/api/postman-collection`) is imported into Postman and tested as follows.

- Import the collection; it appears as PageTurner Online Bookstore API.
- Set the collection variable `baseUrl`: `http://localhost:5050/api` locally, or the Render URL followed by `/api`.
- Run Health Check and expect HTTP 200 with status ok.
- Run Auth > Login User, copy the token into the `authToken` variable; protected requests use Bearer `{{authToken}}`.
- Test Books (list, search), Cart and Wishlist requests.
- Log in as admin and run Create Book; repeating it with a user token returns HTTP 403.

16. Deployment on Render

16.1 Steps

#	Step
1	Push the code to GitHub after confirming that <code>.env</code> is not staged (<code>git add .</code> , <code>git status</code> , <code>git commit</code> , <code>git push</code>).
2	On Render choose New + > Web Service and connect the GitHub repository.

#	Step
3	Settings: Branch main, Runtime Node, Build Command npm install, Start Command npm start, Instance Type Free.
4	Add environment variables: NODE_ENV, JWT_SECRET, and optionally MONGO_URI, RAZORPAY_KEY_ID and RAZORPAY_KEY_SECRET.
5	Click Create Web Service; after the log shows Live, open the URL and verify the dashboard, /api-docs and /api/health.

If MONGO_URI is left empty the built-in MongoMemoryServer is used and the 15 curated books are seeded on start.

16.2 Screenshot

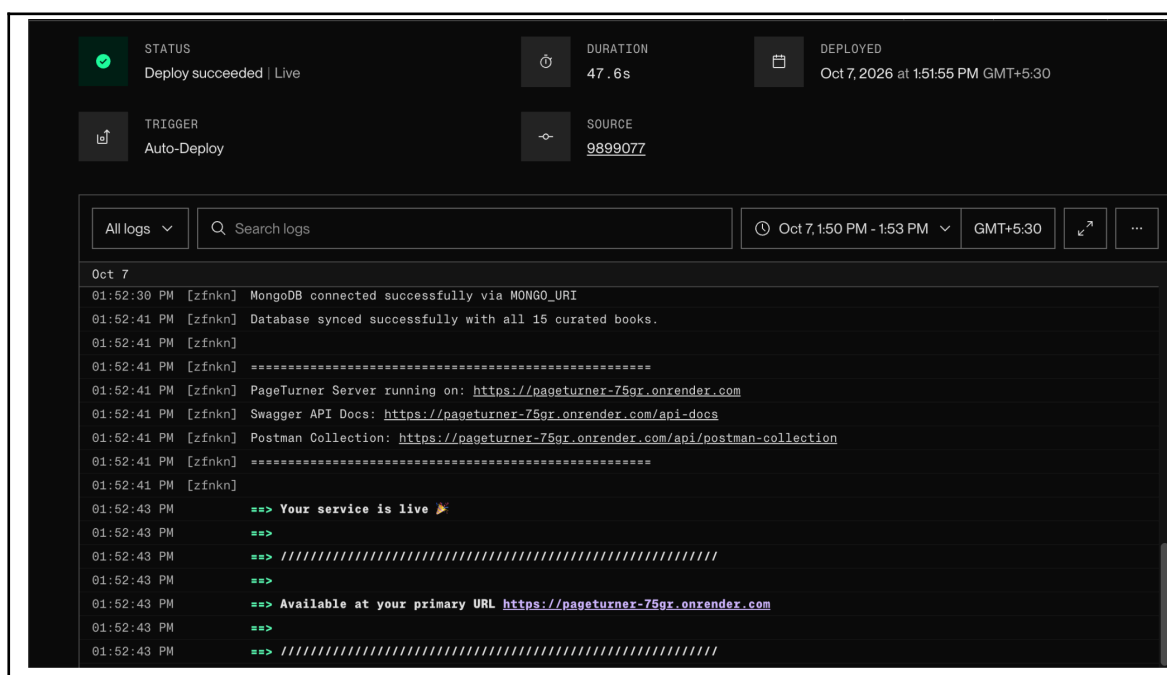


Figure 16.1 — Render dashboard showing the live PageTurner service

17. Testing and Verification Plan

Each objective is verified with a test action and an expected result.

#	Objective	Test action	Expected result
1	Server health	GET /api/health	HTTP 200, status ok
2	Catalogue	GET /api/books, search with keyword	Books with price, rating and cover; matching results only
3	Registration	POST /api/auth/register with a new email	User created; duplicate email rejected
4	Login	POST /api/auth/login	JWT token returned
5	Protected access	Call GET /api/cart without a token	Request rejected as unauthorised
6	Cart	POST /api/cart then GET /api/cart	Book appears with quantity
7	Wishlist	POST then GET /api/wishlist	Book listed; removal works
8	Order	POST /api/orders with address	Order created with Razorpay order details

#	Objective	Test action	Expected result
9	Payment verification	POST /api/orders/verify	Valid signature marks order paid; invalid returns 400
10	Reviews	POST /api/reviews as user and admin	User needs a purchase; admin can post directly
11	Admin access	Create Book with admin token, then user token	Admin succeeds; user gets HTTP 403
12	Cover upload	Create a book with an image file	Image stored in Firebase or public/uploads/

The results above are the expected behaviour of the application; record the observed results and attach matching screenshots when the tests are run.

18. Results and Conclusion

18.1 Findings

Expected outcome	How it was achieved	Evidence
Secure user accounts	bcrypt hashing and JWT tokens	Sections 10, 14
Role-based access	protect and adminOnly middleware	Sections 10, 15
Catalogue, cart and wishlist	Express routes with Mongoose models	Sections 7, 13
Verified payments	Razorpay HMAC-SHA256 signature check	Section 12
Cover upload with fallback	Multer with Firebase or local storage	Section 11
Documentation and deployment	Swagger, Postman and Render with Atlas	Sections 9, 14, 15, 16

18.2 Limitations

- Payments run in test mode or sandbox mode and no real money is processed.
- The free Render instance may sleep when idle, and the in-memory database loses data when the server restarts.
- Only two roles (user and admin) are available.
- The Swagger form does not show the file field for cover upload; the admin web UI is used for that demo.

18.3 Future enhancements

- Use a permanent MongoDB Atlas database in every environment.
- Add order tracking, email notifications and refunds.
- Add book recommendations and reading lists.
- Add automated tests and a CI pipeline.

18.4 Conclusion

PageTurner meets the objectives of the project. It provides a complete online bookstore with secure registration and login, role-based administration, a cart, wishlist and reviews, verified Razorpay payments, cover image upload with cloud and local options, Swagger documentation, a Postman test suite and a live Render deployment. The project shows how Node.js, Express.js, MongoDB, JWT, bcrypt, Multer, Firebase and Razorpay work together in a resilient, evaluation-ready full-stack application.